
Projet : Donjon

Semestre printemps 2025-2026

1 Introduction

Le but de ce projet est de développer un jeu simplifié de type *Dungeon Explorer*¹, un jeu d'exploration dans lequel le joueur se déplace dans un donjon composé de salles, ramasse des objets, gère son inventaire, et combat des ennemis. D'un point de vue pédagogique, le principal objectif est de vous faire progresser dans votre apprentissage du langage C.

2 Consignes et modalités d'évaluation

2.1 Intégrité académique



Le travail doit être réalisé **individuellement**. Vous pouvez discuter du sujet avec vos camarades mais il est strictement interdit de copier-coller du code. L'équipe pédagogique utilise des outils de mesure de similarité de codes sources. Tout plagiat sera sévèrement sanctionné.

Toute utilisation d'une **IA générative** devra être **explicitement mentionnée** – conformément à la charte des usages des IAG à l'UT². Vous devrez aussi explicitement indiquer les parties de code générées par l'IA (et, le cas échéant, être capable d'expliquer précisément ce que ce code généré effectue). La notation en tiendra compte.

2.2 Évaluation

Le travail que vous allez produire sera évalué :

- à partir d'un dépôt sur Moodle que vous devrez effectuer **au plus tard le mercredi 20 mai 2026 à 23h59** (n'attendez pas le dernier moment...) ; quelle que soit la raison, si le **dépôt est vide** après cette date, **vous aurez 0** ; il est donc conseillé de déposer votre projet avant, même s'il n'est pas fini ; mais la version déposée doit pouvoir être compilée sans erreur ;
- à partir d'un oral qui se déroulera la semaine du **25 mai 2026** (cf. emploi du temps pour le créneau précis de votre groupe).

2.3 Barème

Le projet comporte quatre niveaux de difficulté croissante, numérotés de 0 à 3. La réalisation complète et exacte de chaque niveau donne la possibilité d'obtenir une certaine note maximale, comme cela est indiqué dans le tableau suivant :

Niveau	Note maximale
0	8
1	12
2	14
3	20

1. [https://en.wikipedia.org/wiki/Dungeon_Explorer_\(1989_video_game\)](https://en.wikipedia.org/wiki/Dungeon_Explorer_(1989_video_game))

2. https://www.univ-tlse3.fr/medias/fichier/charte-du-bon-usage-des-ia-generatives-a-l-universite_1755517998722-pdf



Pour commencer un niveau, il faut avoir complètement réalisé le niveau précédent. Un niveau est considéré comme réalisé quand toutes ses fonctionnalités sont programmées et que le programme correspondant au niveau s'exécute sans erreur.

Attention : si le projet que vous avez déposé sur Moodle ne peut pas être **compilé sans erreur** (sur les machines des **salles** de TP) en utilisant la commande **make**, vous aurez automatiquement la note 0. Vous pouvez accéder à une salle du U3 en empruntant une carte d'accès auprès du secrétariat de la licence. Un accès distant est aussi possible (voir documentation sur Moodle à ce sujet). N'attendez pas le dernier moment pour vérifier que votre projet compile sur une machine des salles de TP.

2.4 Dépôt sur Moodle

Vous déposerez sur Moodle un **seul fichier archive** au format ZIP qui doit être produit en suivant les directives suivantes.

1. Créez un répertoire dont le nom doit suivre le modèle :

NOM-PRENOM-NUMETU

où :

- NOM, PRENOM et NUMETU doivent être remplacés, respectivement et dans cet ordre, par votre nom de famille en majuscules, votre prénom en majuscules et votre numéro d'étudiant (8 chiffres) ;
- n'apparaît aucune lettre accentuée ;
- n'apparaît aucun espace ni apostrophe, les éventuels espaces et apostrophes dans votre nom ou votre prénom devant être remplacés par des tirets - (signe moins).

2. Placez dans ce répertoire :

- tous les fichiers sources de votre projet, c'est-à-dire les fichiers d'extensions `.h` et `.c` ;
- un fichier de description des dépendances de nom `Makefile` ;
- un fichier de texte (obtenu avec un simple éditeur de texte) de nom `niveau-n.txt` où vous remplacerez n par le numéro du niveau le plus haut que vous avez réalisé (0, 1, 2 ou 3) ; ce fichier pourra être vide ou contenir d'éventuelles informations destinées à expliquer les choix que vous aurez faits.

Vos fichiers sources doivent être indentés et contenir des commentaires permettant de bien comprendre votre travail. Aucun autre fichier (objets, exécutable, données, résultats, etc.) ne doit être présent dans le répertoire.

3. Créez une archive au format ZIP de ce répertoire ; cette archive doit porter le même nom que le répertoire auquel est ajoutée l'extension `.zip` ; pour produire cette archive, dans le terminal, vous pouvez vous placer dans le répertoire parent du répertoire contenant vos fichiers sources et utiliser la commande :

`zip NOM-PRENOM-NUMETU.zip NOM-PRENOM-NUMETU/*`

Respectez bien toutes ces consignes car des tests et des vérifications automatiques seront effectués sur votre projet.

2.5 Oral

Au moment de l'oral, vous devrez être prêt à :

- faire fonctionner votre programme ;
- répondre aux questions de l'enseignant ;
- effectuer des tests ou des modifications demandées par l'enseignant.

2.6 Consignes communes aux différents niveaux

2.6.1 Noms des fichiers et fonctions demandées

Pour chaque niveau, les fichiers d'en-tête (.h) correspondant aux différents modules vous sont imposés. Les déclarations des fonctions (leurs noms et leurs signatures), les définitions des structures et des types manipulés **ne doivent pas être modifiés**. Vous pouvez bien entendu ajouter des fonctions de votre côté mais en aucun cas vous devez modifier la déclaration des fonctions demandées dans cet énoncé. Votre travail sera notamment évalué sur la base de tests automatiques, la modification des fichiers en-têtes rendra impossible leur compilation, et **votre travail sera évalué avec la note 0**.

2.6.2 Aide à la mise au point

Afin de vérifier que vos programmes n'ont pas de « fuite » mémoire, c'est-à-dire gèrent correctement l'allocation dynamique et la libération des zones en mémoire, un outil d'analyse dynamique peut vous aider. Par exemple, avec l'exécutable `test_application`, vous pouvez :

- sous Linux, utiliser Valgrind³ avec la commande : `valgrind ./test_application`
- sous MacOS, utiliser Leaks avec la commande : `leaks --atExit -- ./test_application`

Dans les deux cas, vérifiez le nombre de « leaks » à la fin de l'affichage produit.

Il existe également des outils (*code sanitizers*⁴) qui détectent, au moment de l'exécution du programme, les erreurs d'accès à la mémoire comme le débordement d'un tableau. Sur les machines des salles de TP, quand vous invoquez gcc, vous pouvez ajouter l'option `-fsanitize=address` qui permettra l'affichage d'informations en cas d'erreur d'accès à la mémoire lors de l'exécution d'un programme.

3 Niveau 0 : modules inventaire, joueur, salle, donjon

3.1 Module inventaire

Le module `inventaire` permet de gérer une liste d'objets, chaque objet étant représenté par la structure de données privée suivante :

```
// Structure privée d'un objet de l'inventaire
struct sItem {
    char nom[NOM_MAX]; // chaîne de caractères, taille statique NOM_MAX=32
    int quantite;
    struct sItem *suiv;
};
```

Et l'inventaire sera une liste chaînée d'objets, en utilisant la structure de données privée :

```
// Structure privée d'un inventaire (liste chaînée)
struct sInventaire {
    struct sItem *debut;
};
```

Le type public `tInventaire` sera défini dans le fichier `inventaire.h` de la façon suivante :

```
// Type public représentant un inventaire (pointeur vers structure privée)
typedef struct sInventaire *tInventaire;
```

3. <https://fr.wikipedia.org/wiki/Valgrind>

4. https://en.wikipedia.org/wiki/Code_sanitizer

Les fonctions à écrire (publiques, déclarées dans le .h) sont les suivantes :

- `tInventaire InventaireCreer(void)`; crée un inventaire vide. La structure doit être allouée dynamiquement. La fonction doit renvoyer l'inventaire créé ou NULL en cas de problème d'allocation mémoire.
- `void InventaireVider(tInventaire inv)`; retire et libère tous les objets de l'inventaire. Après cette fonction, l'inventaire doit être vide (un pointeur vers une structure dont le champ `debut` vaut NULL).
- `void InventaireLiberer(tInventaire *pinv)`; libère tout l'espace mémoire occupé par l'inventaire. Après cette fonction, l'inventaire doit être détruit et le pointeur vers l'inventaire doit être positionné à NULL.
- `int InventaireAjouter(tInventaire inv, const char *nom, int qte)`; ajoute l'objet `nom` à l'inventaire avec la quantité `qte`. Si l'objet n'est pas présent, il est créé en début de liste, sinon sa quantité est mise à jour. La fonction renvoie 1 si l'ajout a pu se faire, 0 sinon.
- `int InventaireRetirer(tInventaire inv, const char *nom, int qte)`; supprime `qte` quantité de l'objet `nom` (il est impossible de retirer plus que disponible). Si la quantité passe à 0, l'objet est supprimé de l'inventaire. La fonction renvoie 1 si le retrait a pu se faire, 0 sinon.
- `int InventaireQuantite(const tInventaire inv, const char *nom)`; renvoie la quantité associée à l'objet `nom` dans l'inventaire. Si l'objet n'est pas présent ou si les paramètres sont invalides, la fonction renvoie 0.
- `int InventaireCompter(const tInventaire inv)`; renvoie le nombre d'objets distincts présents dans l'inventaire.
- `char *InventaireVersChaine(const tInventaire inv)`; construit et renvoie une chaîne de caractères représentant le contenu de l'inventaire. Chaque objet est représenté sous la forme `nom(quantite)` et les objets sont séparés par une virgule et un espace (", "). Si l'inventaire est vide, la chaîne "vide" est renvoyée. La chaîne est allouée dynamiquement (on suppose une quantité max à 999) et doit être libérée par l'appelant quand il n'en a plus besoin. La fonction renvoie NULL en cas d'erreur d'allocation mémoire. Exemple de chaîne de caractères produite : "bandage(2),_potion(1)"

Test du module inventaire

Créez le fichier source `test_inventaire.c` contenant la fonction principale `main` afin de tester toutes les fonctions du module `inventaire` (création d'un inventaire, ajout et retrait d'objets, test de la quantité, affichage du contenu, et destruction de l'inventaire).

Effectuez la compilation séparée grâce à l'utilitaire `make`. Pour cela, **écrivez un fichier de description des dépendances Makefile** et lancez les compilations en tapant : `make test_inventaire`

3.2 Module joueur

Le module `joueur` permet de gérer le joueur du jeu. Il est représenté par la structure de données privée suivante :

```
// Structure privée d'un joueur
struct sJoueur {
    int x, y;           // position du joueur dans le donjon
    int pv, pvMax;      // points de vie actuels / maximum
    int atk, def;       // attaque et défense
    tInventaire inv;    // inventaire
};
```

Le type public `tJoueur` sera défini dans le fichier `joueur.h` de la façon suivante :

```
// Type public (pointeur vers structure privée)
typedef struct sJoueur *tJoueur;
```

Les fonctions à écrire (publiques, déclarées dans le `.h`) sont les suivantes :

- `tJoueur JoueurCreer(int x, int y)`; crée un joueur positionné aux coordonnées (x, y) dans le donjon. La structure est allouée dynamiquement et ses attributs sont initialisés avec des valeurs par défaut (pv 20, pvMax 20, atk 5, def 1). Un inventaire vide est également créé et associé au joueur. La fonction renvoie le joueur créé ou NULL en cas de problème d'allocation mémoire.
- `void JoueurLiberer(tJoueur *pj)`; libère tout l'espace mémoire occupé par le joueur. L'inventaire associé est également libéré. Le pointeur sur le joueur est positionné à NULL après destruction.
- Fonctions de lecture des champs de la structure privée :
 - `void JoueurPosition(const tJoueur j, int *px, int *py)`;
 - `int JoueurPV(const tJoueur j)`;
 - `int JoueurPVMax(const tJoueur j)`;
 - `int JoueurAttaque(const tJoueur j)`;
 - `int JoueurDefense(const tJoueur j)`;
 - `tInventaire JoueurInventaire(const tJoueur j)`;
- Fonctions de mise à jour de la position et des points de vie :
 - `void JoueurMajPos(tJoueur j, int x, int y)`;
 - `void JoueurMajPV(tJoueur j, int pv)`;

3.3 Module salle

Le module `salle` est utilisé pour manipuler les salles du donjon. Une salle est représentée par la structure de données privée suivante :

```
// Structure privée d'une salle
struct sSalle {
    int existe;           // 0 = mur, 1 = salle
    int visitee;          // 0 = non, 1 = oui
    char *desc;           // description (allouée dynamiquement)
    tInventaire objets;   // liste d'objets au sol sous la forme d'un inventaire
};
```

Le type public `tSalle` sera défini dans le fichier `salle.h` de la façon suivante :

```
// Type public (pointeur vers structure privée)
typedef struct sSalle *tSalle;
```

Les fonctions à écrire (publiques, déclarées dans le `.h`) sont les suivantes :

- `tSalle SalleCreerMur(void)`; crée une salle correspondant à un mur. La structure est allouée dynamiquement et initialisée de sorte que la salle n'existe pas (`existe` à 0). Les autres champs sont initialisés à des valeurs neutres (0 ou NULL). La fonction renvoie la salle créée ou NULL en cas de problème d'allocation mémoire.

- `tSalle SalleCreerVide(void)`; crée une salle vide (accessible) dans le donjon. La structure est allouée dynamiquement, marquée comme existante (`existe` à 1) et non visitée. Une description par défaut⁵ est allouée dynamiquement et copiée dans la salle. Un inventaire vide est également créé pour représenter les objets présents au sol. La fonction renvoie la salle créée ou `NULL` en cas de problème d'allocation mémoire (toutes les allocations intermédiaires sont correctement libérées en cas d'échec).
- `void SalleLiberer(tSalle *ps)`; libère tout l'espace mémoire occupé par une salle. Si la salle correspond à un mur, seule la structure est libérée. Sinon, tous les champs alloués dynamiquement sont libérés. Le pointeur sur la salle est positionné à `NULL` après destruction.
- Fonctions de lecture des champs de la structure privée :
 - `int SalleExiste(const tSalle s)`;
 - `int SalleEstVisitee(const tSalle s)`;
 - `const char *SalleDescription(const tSalle s)`;
 - `tInventaire SalleObjets(const tSalle s)`;
- Fonctions de mise à jour :
 - `int SalleMajDescription(tSalle s, const char *desc)`; met à jour la description d'une salle existante. La description courante est libérée puis remplacée par une copie de la chaîne `desc`, allouée dynamiquement. La fonction renvoie 1 si la mise à jour a pu être effectuée, 0 en cas de problème.
 - `void SalleVisiter(tSalle s)`;

3.4 Module donjon

Le module `donjon` est l'implémentation du donjon du jeu. Il est représenté par la structure de données privée suivante :

```
// structure privée donjon
struct sDonjon {
    int w, h;          // taille du donjon (w largeur, h hauteur)
    tSalle *cases;     // tableau dynamique de taille w * h des salles du donjon
};
```

Le type public `tDonjon` sera défini dans le fichier `donjon.h` de la façon suivante :

```
// Type public (pointeur vers structure privée)
typedef struct sDonjon *tDonjon;
```

Les fonctions à écrire (publiques, déclarées dans le `.h`) sont les suivantes :

- `tDonjon DonjonCreer(int w, int h)`; crée un donjon de dimensions $w \times h$. La structure est allouée dynamiquement, ainsi que `cases`, le tableau à **une dimension** pour contenir les salles du donjon. Les salles ne sont pas initialisées par cette fonction. La fonction renvoie le donjon créé ou `NULL` en cas de paramètres invalides ou de problème d'allocation mémoire.
- `void DonjonLiberer(tDonjon *pd)`; libère tout l'espace mémoire occupé par un donjon. Chaque salle du tableau est libérée individuellement, puis le tableau de cases est libéré, ainsi que la structure du donjon. Le pointeur sur le donjon est positionné à `NULL` après destruction.

5. Par exemple : "Une salle sans particularite."

- Fonctions de lecture des champs de la structure privée :
 - `int DonjonH(const tDonjon d);`
 - `int DonjonW(const tDonjon d);`
 - `tSalle DonjonSalle(const tDonjon d, int x, int y);` renvoie la salle située aux coordonnées (x, y) dans le donjon, ou NULL en cas d'erreur. La première salle a pour coordonnées $(0,0)$. Il faudra s'assurer que les coordonnées sont bien comprises dans les dimensions du donjon. Dans un tableau à une dimension, la salle située aux coordonnées (x, y) se trouve à l'indice $y \times w + x$.
- `int DonjonCharger(const char *fichier, tDonjon *dj, int *departX, int *departY);` charge un donjon à partir d'un fichier texte. Le fichier doit contenir d'abord les dimensions du donjon (nombre de colonnes puis nombre de lignes), puis la grille des cases, où '#' représente un mur, '.' une salle vide et '@' une salle vide correspondant à la position de départ du joueur (cette salle devra donc être marquée visitée). La fonction crée dynamiquement le donjon, initialise ses salles à partir de la grille, puis traite les directives optionnelles permettant de préciser la description des salles (DESC <x> <y> <description de la salle (x, y)>) et les objets présents dans les salles (ITEM <x> <y> <nom> <quantite>). Si une ligne lue ne correspond pas une directive connue, la lecture doit se poursuivre, ligne par ligne, jusqu'à la fin du fichier. Les coordonnées de départ du joueur sont délivrées en sortie via `departX` et `departY`, et le donjon via `dj`. La fonction renvoie 1 si le chargement a réussi, 0 sinon. Des exemples de fichier de description d'un donjon sont donnés en pièces jointes au projet sur Moodle. Exemple :

```

8 4
..@...#..
..#...#..
..#...#..
.....
DESC 2 0 La salle de depart.
DESC 1 1 Une alcove poussiéreuse.
DESC 3 1 Un couloir étroit, des pierres humides.
ITEM 3 1 potion 2
ITEM 6 2 cle 1

```

Test des modules joueur, salle et donjon

Créez le fichier source `test_donjon.c` contenant la fonction principale `main` afin de tester la création du donjon et son affichage via le module `ui.h` que nous vous fournissons, et qui est décrit en annexe (section 7). Cette fonction principale de test se contentera d'appeler les fonctions : `DonjonCharger`, `JoueurCreer`, `UI_Creer`, puis enfin `UI_Afficher` pour tester l'affichage de l'interface utilisateur. Toutes les ressources devront être libérées avant la terminaison du programme de test, et vous vérifierez l'absence de fuite mémoire.

Vous mettrez à jour le `Makefile` précédent pour ajouter la cible `test_donjon`, permettant de lancer les compilations et la production de l'exécutable en tapant : `make test_donjon`

Important : vous ajouterez dans les options de compilation (variable `CFLAGS`) l'option `-DNIV01` qui permet de définir la constante symbolique `NIV01` qui est utilisée par le module `ui` qui vous est donné.

4 Niveau 1 : création du jeu et exploration du donjon

4.1 Module jeu

4.1.1 Fonctions publiques

Le module jeu contiendra la structure publique suivante :

```
struct sJeu {  
    tDonjon d; // donjon  
    tJoueur j; // joueur  
    tUI ui;    // interface utilisateur  
    int fini;  // fin du jeu si la valeur est à 1  
};
```

Et les trois fonctions publiques :

- `struct sJeu *JeuCreer(const char *fichierDonjon)`; crée le donjon à partir du nom de fichier `fichierDonjon`, crée un joueur, et crée l'interface utilisateur. La structure `sJeu` sera allouée dynamiquement, initialisée et renvoyée par la fonction, ou `NULL` en cas de problème lors de la création d'un des éléments. Le message "Bienvenue ! Tapez help pour avoir la liste des commandes" devra également être positionné sur l'interface via la fonction `UI_DefinirMessage`.
- `void JeuLiberer(struct sJeu *g)`; libère toutes les ressources associées à la structure pointée par `g`, ainsi que la structure elle-même.
- `void JeuJouer(struct sJeu *g)`; implémente la boucle principale du jeu. Tant que le jeu n'est pas fini, cette boucle devra : afficher l'interface utilisateur (`UI_Afficher`), lire la commande saisie (`UI_LireLigne`), et l'exécuter (via la fonction privée `execute_command`, cf. plus bas). Attention, la commande (chaîne de caractères) retournée par la fonction `UI_LireLigne` devra être détruite après son utilisation (cf. documentation associée à cette fonction).

4.1.2 Fonctions privées

Les fonctions privées (déclarées en `static`) suivantes devront être implémentées :

- `void execute_command(struct sJeu *g, char *ligne_cmd)`; analyse la chaîne de caractères passée en paramètre (`ligne_cmd`) et appelle la fonction adéquate (parmi celles décrites ci-après). Les commandes suivantes devront être considérées :
 - "quit" ou "q" : quitter le jeu ;
 - "help" ou "h" : affiche l'aide ;
 - "n" ou "e" ou "s" ou "w" : déplace le joueur d'une case vers le nord, est, sud ou ouest ;
 - "take <obj>" : prend l'objet <obj> présent de la salle ;
 - "drop <obj>" : dépose l'objet <obj> dans la salle.
- `void cmd_help(struct sJeu *g)`; affiche une ligne d'aide au joueur du type :
"Commandes: n/e/s/w | take <obj> | drop <obj> | quit | help"
- `int cmd_move(struct sJeu *g, int dx, int dy)`; déplace le joueur dans le donjon de `dx` cases horizontalement et de `dy` cases verticalement. Si le déplacement est impossible, un message d'erreur devra être positionné (via `UI_DefinirMessage`), sinon la position du joueur sera mise à jour, et la nouvelle salle sera marquée visitée. En cas de déplacement réussi, la fonction renverra 1, 0 sinon.

- `void cmd_take(struct sJeu *g, const char *name);` prélève l'objet `name` (une seule quantité) présent dans la salle et l'ajoute à l'inventaire du joueur. Si l'objet n'existe pas dans la salle, ou si une erreur survient lors de l'ajout à l'inventaire, un message d'erreur devra être affiché sur l'interface utilisateur.
- `void cmd_drop(struct sJeu *g, const char *name);` retire l'objet `name` (une seule quantité) de l'inventaire du joueur, et le dépose dans la salle. Si l'objet n'existe pas dans l'inventaire du joueur, ou si une erreur survient lors de l'ajout dans la salle, un message d'erreur devra être affiché sur l'interface utilisateur.

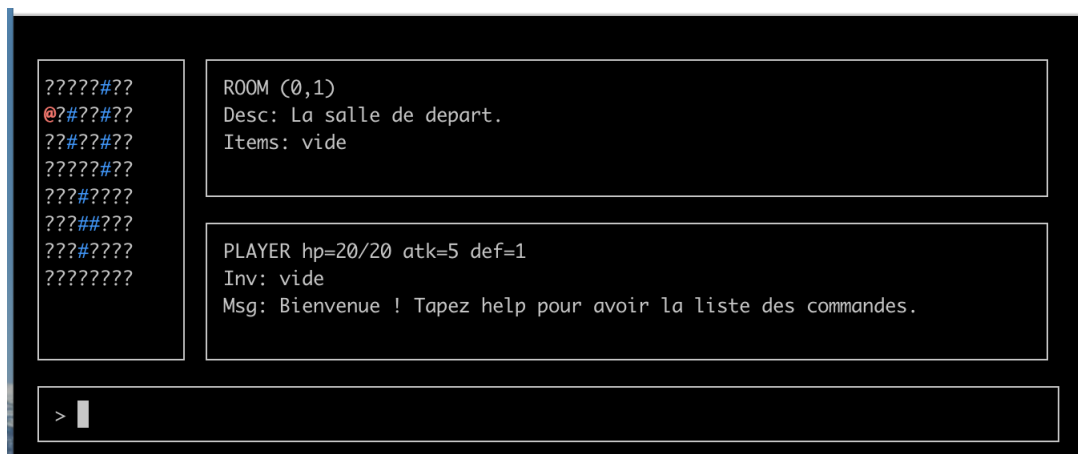
4.2 Fonction principale

Ecrivez la fonction principale du jeu dans un fichier `main.c` qui ne s'appuiera que sur le module `jeu` en utilisant les fonctions `JeuCreer`, `JeuJouer` et `JeuLiberer`. Le nom du fichier texte contenant la description du donjon devra être passé en paramètre du programme (utilisation de `argc/argv`).

Vous complétez le `Makefile` précédent en ajoutant la cible `donjon` pour produire l'exécutable (ce sera la première cible pour, par défaut, compiler l'application). **Important** : comme pour le niveau précédent vous ajouterez dans les options de compilation (variable `CFLAGS`) l'option `-DNIV01`.

Exemple :

```
$ make
$ ./donjon donjon64.txt
```



5 Niveau 2 : combat avec des ennemis

5.1 Module ennemi

Un ennemi est représenté par la structure de données privée suivante :

```
// Structure privée d'un ennemi
struct sEnnemi {
    char nom[NOM_MAX]; // NOM_MAX est à 32
    int pv, pvMax, atk, def;
};
```

Le type public `tEnnemi` sera défini dans le fichier `ennemi.h` de la façon suivante :

```
// Type public (pointeur vers structure privée)
typedef struct sEnnemi *tEnnemi;
```

Les fonctions à écrire (publiques, déclarées dans le .h) sont les suivantes :

- `tEnnemi EnnemiCreer(const char *nom, int pvMax, int atk, int def)`; crée un ennemi par allocation dynamique de la structure. Tous les champs de la structure sont initialisés grâce aux paramètres de la fonction. La fonction renvoie l'ennemi créée ou NULL en cas de problème d'allocation mémoire.
- `void EnnemiLiberer(tEnnemi *pe)`; libère l'espace mémoire occupé par un ennemi. Le pointeur sur l'ennemi est positionné à NULL après destruction.
- Fonctions de lecture des champs de la structure privée :
 - `char *EnnemiNom(const tEnnemi e)`;
 - `int EnnemiPV(const tEnnemi e)`;
 - `int EnnemiPVMax(const tEnnemi e)`;
 - `int EnnemiAttaque(const tEnnemi e)`;
 - `int EnnemiDefense(const tEnnemi e)`;
 - `int EnnemiEstVivant(const tEnnemi e)`; renvoie 1 si les points de vie sont > 0.
- Fonctions de mise à jour :
 - `int EnnemiSubirAttaque(tEnnemi e, const tJoueur j)`; met à jour les points de vie de l'ennemi et retourne les dégâts infligés (le nombre de points perdus). Les dégâts sont calculés de la façon suivante : `degats = max(1, JoueurAttaque(j) - EnnemiDefense(e))`. Les points de vie de l'ennemi après l'attaque ne peuvent pas être inférieur à 0.
 - `int EnnemiAttaquerJoueur(const tEnnemi e, tJoueur j)`; met à jour les points de vie du joueur et retourne les dégâts infligés (le nombre de points perdus). Les dégâts sont calculés de la façon suivante : `degats = max(1, EnnemiAttaque(e) - JoueurDefense(j))`. Les points de vie du joueur après l'attaque ne peuvent pas être inférieur à 0.
 - `void EnnemiMajPV(tEnnemi e, int pv)`; modifie les points de vie de l'ennemi à la valeur pv.

5.2 Mise à jour du module salle

Une salle peut contenir un ennemi. Vous mettrez donc à jour la structure `struct sSalle` en ajoutant le champ `tEnnemi ennemi`;

Cet ajout implique les modifications suivantes dans le module `salle` :

- Initialisation de l'ennemi à NULL dans les fonctions `SalleCreerMur` et `SalleCreerVide`.
- Libération de la mémoire associée à l'ennemi dans `SalleLiberer` (s'il est présent dans la salle).
- Ajout de deux fonctions :
 - `tEnnemi SalleEnnemi(tSalle s)`; pour la lecture de l'ennemi;
 - `void SalleAjouterEnnemi(tSalle s, const tEnnemi e)`; pour l'ajout dans la salle.

5.3 Mise à jour du module donjon

Modifiez la fonction `DonjonCharger` pour permettre de traiter, dans le fichier texte de description du donjon, la directive optionnelle :

```
ENEMY <x> <y> <nom> <pv> <atk> <def>
```

Exemple : la ligne `ENEMY 1 3 rat 6 2 0` indique un ennemi « rat » présent dans la salle de coordonnées $x = 1$, $y = 3$, avec 6 points de vie, une capacité d'attaque de 2 et de défense de 0.

5.4 Mise à jour du module jeu

Vous allez ajouter la possibilité de combattre des ennemis dans le jeu. Pour cela, ajoutez la commande "fight" ou "f" dans la fonction privée `execute_command`. Cette commande appellera une nouvelle fonction privée :

- `void cmd_fight(struct sJeu *g);` lance une attaque sur l'ennemi présent dans la salle où se situe le joueur. Si aucun ennemi vivant est présent dans la salle, un message d'erreur sera positionné sur l'interface (`UI_DefinirMessage`). Dans le cas contraire, un combat se déroule en deux étapes :
 1. Le joueur attaque l'ennemi ;
 2. Si l'ennemi est toujours vivant, il inflige une riposte au joueur.

Le résultat du combat doit être affiché sur l'interface utilisateur (dégâts infligés et reçus, mort éventuelle de l'ennemi ou du joueur). Exemple d'affichage :

"Msg: Vous infligez 3 degats. L'ennemi riposte (5). Vous etes mort.".

5.5 Tests des combats

Pour tester ce niveau, vous pouvez combattre des ennemis présents dans les donjons fournis en exemple. Vérifiez bien que les points de vie de l'ennemi et du joueur sont correctement mis à jour à chaque attaque.

6 Niveau 3 : Sauvegarde et recherche automatique du trésor

6.1 Itération dans l'inventaire

L'encapsulation des données de l'inventaire masque la liste chaînée utilisée pour conserver les objets de l'inventaire. Nous devons donc passer par une fonction publique pour consulter tous les objets de la collection sans exposer sa structure interne (on parle du patron de conception « Itérateur »).

Pour cela vous allez ajouter la fonction suivante dans le module `inventaire` :

- `int InventaireVisiter(
 const tInventaire inv,
 int (*visiter)(const char *nom, int quantite, void *contexte),
 void *contexte);`

Cette fonction permet de parcourir tous les objets de l'inventaire et d'appeler, pour chacun d'eux, la fonction `visiter` (déclarée ici comme un pointeur de fonction). Le paramètre `contexte` est un pointeur générique permettant de transmettre des informations supplémentaires à la fonction `visiter`. Lors du parcours de l'inventaire, si la fonction `visiter` retourne 0, alors il faut arrêter immédiatement le parcours et retourner 0, sinon continuer avec l'objet suivant. La fonction retourne 1 si tous les objets ont été parcourus.

La fonction `InventaireVisiter` sera utilisée par la suite par les modules `save` et `explore`.

Note : les pointeurs de fonctions sont vus en Cours/TD dans le chapitre 13 sur la généricité.

6.2 Module de sauvegarde save

Ce module ajoute la possibilité de sauvegarder la partie dans un fichier binaire, et de recharger une partie déjà sauvegardée.

Le format du fichier binaire sera le suivant :

- **L'en-tête** (*header*) : 4 caractères "DNG1" et 3 entiers (`int`) : l'entier 1 pour la version du jeu, puis la largeur et la hauteur du donjon.
- **Le joueur** : 3 entiers (`int`) avec les valeurs de `x`, `y` et `pv`.
- **L'inventaire** : 1 entier (`int`) pour le nombre d'objets dans l'inventaire. Si ce nombre est supérieur à 0, alors pour chaque objet : son nom (tableau statique) et sa quantité (`int`).
- **Les salles** du donjon. Pour chaque salle :
 - un 1 (`unsigned char`) si la salle existe, 0 sinon (un mur) ;
 - si la salle existe :
 - un 1 (`unsigned char`) si elle a été visitée, 0 sinon ;
 - les objets présents dans la salle, stockés de la même manière que l'inventaire du joueur ;
 - un 1 (`unsigned char`) si un ennemi est dans la salle, 0 sinon. Si un ennemi est présent, alors il y aura son nom (tableau statique) et 4 entiers (`int`) : `pv`, `pvMax`, `atk` et `def`.

Nous ne sauvegardons pas les descriptions des salles, celles-ci sont chargées au début via le fichier texte de description du donjon.

Le module `save` devra comporter les deux fonctions suivantes :

- `int SaveEcrire(const char *fichier, tDonjon d, tJoueur j)` ; écrit dans le fichier en mode binaire l'état courant du donjon et du joueur. Le format devra respecter la spécification donnée ci-dessus. Pour écrire les objets de l'inventaire et des salles, vous utiliserez une fonction d'écriture d'un seul objet (par exemple `int ecrire_objet(const char *nom, int qte, void *fich)`) qui sera passée en paramètre de la fonction `InventaireVisiter` définie dans la section précédente. La fonction renvoie 1 si l'écriture a pu se faire sans erreur, 0 sinon.
- `int SaveLire(const char *fichier, tDonjon d, tJoueur j)` ; lit dans le fichier binaire une partie précédemment sauvegardée. La lecture respectera le même format que celui utilisé pour l'écriture. Si la taille du donjon lu n'est pas la même que celle du jeu, ou la disposition n'est pas la même (incohérence salle/mur) alors la fonction renverra 0 et un message d'erreur sera affiché. La fonction renvoie 1 si le chargement a pu se faire sans erreur. **Notez bien que le chargement de la partie doit venir remplacer tout l'état courant du jeu** (par exemple, si des objets sont déjà présents dans une salle, il faut commencer par les retirer avant d'ajouter ceux de la partie sauvegardée).

Intégration de la sauvegarde au module jeu

Ajoutez les deux commandes "save <fichier>" et "load <fichier>" au menu de l'utilisateur. Ces commandes appelleront respectivement les fonctions `SaveEcrire` et `SaveLire`.

6.3 Module d'exploration explore

Pour cette dernière partie, on vous demande d'implémenter une recherche automatique du trésor dans le donjon à l'aide d'un algorithme de parcours en profondeur d'un graphe (ou DFS, pour *Depth-First Search*⁶). On s'intéressera à la **version récursive du parcours en profondeur**.

Un trésor est un objet dont le nom est "tresor". L'exploration doit :

- partir de la position actuelle du joueur,
- visiter les salles accessibles sans traverser les murs,
- s'arrêter dès qu'un trésor est trouvé,
- écrire dans un fichier texte la trace des déplacements.

La fonction publique du module `explore` qui sera appelée par une commande du jeu sera :

- `int ExploreTrouverTrésor(tDonjon d, tJoueur j, const char *fichierTrace);`

Cette fonction devra :

1. Ouvrir et fermer le fichier texte de nom `fichierTrace` en écriture.
2. Allouer une structure de marquage des salles visitées (tableau dynamique de `unsigned char`).
3. Écrire la ligne de départ : `START <x> <y>`
4. Appeler la fonction privée récursive `explorerRec` à partir de la position du joueur.
5. Écrire la position finale : `END <x> <y>` et indiquer si un trésor a été trouvé : `FOUND <0 ou 1>`

Le parcours du donjon se fera par la fonction privée récursive :

- `int explorerRec(tDonjon d, tJoueur j, int x, int y, unsigned char *visited, FILE *f);`

Cette fonction explore le donjon à partir d'une position (x, y). Elle doit suivre les étapes suivantes :

1. Marquer la salle courante dans le tableau `visited` et dans le donjon (pour l'affichage).
2. Mettre à jour la position du joueur.
3. Si la salle contient un objet trésor, renvoyer 1 (succès, arrêter l'exploration). **Pour savoir si la salle contient un trésor, faites appel à la fonction `InventaireVisiter` avec une nouvelle fonction visiteur d'identification du trésor.**
4. Sinon, pour chaque direction parmi Nord, Est, Sud, Ouest :
 - (a) Calculer les coordonnées de la salle voisine.
 - (b) Vérifier que la case est valide (dans le donjon, pas un mur, pas déjà visitée).
 - (c) Si la case est valide : écrire le déplacement dans le fichier (caractères N, E, S ou W) et appeler récursivement `explorerRec` sur cette nouvelle salle.
 - (d) Si l'appel récursif retourne 1, propager ce résultat (return 1), le trésor a été trouvé dans cette direction. Sinon, effectuer un retour arrière (*backtracking*) : écrire le déplacement inverse dans le fichier (ajouter la chaîne "(BACK)" après le caractère de direction) et remettre le joueur à la position précédente.
5. Si toutes les directions ont été testées sans succès, renvoyer 0 (aucun trésor trouvé).

Un exemple de recherche d'un trésor dans un donjon de 4×4 est donné dans la figure 1. Au départ, le joueur est dans la case (1,0), et à l'arrivée, il est dans la case (0,3) où le trésor a été trouvé. Le fichier `trace_tresor.txt` contient la trace du parcours (schématisé par des flèches sur la grille d'arrivée).

6. https://fr.wikipedia.org/wiki/Algorithme_de_parcours_en_profondeur

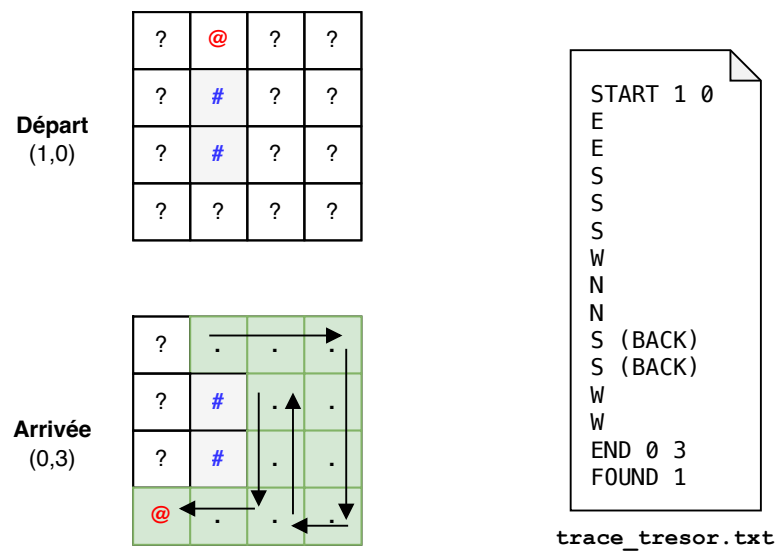


FIGURE 1 – Exemple de parcours en profondeur pour un donjon de 4×4

Intégration de l'exploration au module jeu

Ajoutez une commande "cheat <tracefile>" au menu de l'utilisateur. Cette commande devra appeler la fonction `ExploreTrouverTresor` et positionner un message sur l'interface utilisateur pour indiquer le résultat (par ex. "Tresor trouve ! Trace: `trace_tresor.txt`").

7 Annexe

7.1 Interface ui.h

```
#ifndef UI_H
#define UI_H

#include "donjon.h"
#include "joueur.h"

/* Type public (pointeur vers structure privée) */
typedef struct sUI *tUI;

/* Création de l'interface utilisateur (UI) */
tUI UI_Creer(void);

/* Destruction de l'interface utilisateur */
void UI_Liberer(tUI *pui);

/* Positionne le message qui doit être affiché sur l'interface */
void UI_DefinirMessage(tUI ui, const char *fmt, ...);

/* Affichage de l'interface utilisateur */
void UI_Afficher(tUI ui, tDonjon d, tJoueur j);

/* Lecture d'une ligne saisie par l'utilisateur (une commande à exécuter)
 * La fonction alloue dynamiquement la ligne lue (sans le '\n'),
 * il faudra donc la libérer (free) après utilisation. */
char *UI_LireLigne(tUI ui);

#endif
```

7.2 Implémentations stdio (ui.c) et ncurses (ui_nc.c) du module ui

Nous vous fournissons deux implémentations du module ui :

- ui.c qui utilise la bibliothèque standard pour les entrées/sorties;
- ui_nc.c qui utilise la bibliothèque ncurses⁷ pour une interaction et un affichage « améliorés ». Par exemple, il est possible avec cette version de déplacer le joueur avec les touches flèches. **Pour utiliser cette bibliothèque, vous devez passer l'option -lncurses au compilateur lors de la phase d'édition de liens.**

Vous pouvez prendre l'une ou l'autre des implémentations pour votre programme. La bibliothèque ncurses devrait déjà être installée dans les salles de TP. Sur votre PC personnel, si vous avez Linux Ubuntu par exemple, vous pouvez l'installer via la commande : `sudo apt install libncurses-dev`

7. <https://invisible-island.net/ncurses/>